

Survival Game

Бондарь София 10.3

March 12, 2026

1 Сюжет и мир игры

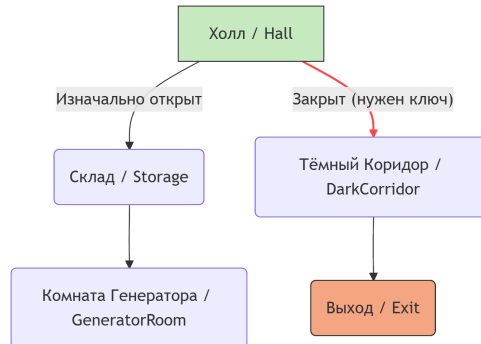
Вы — техник Иван Иванов, очнувшийся в жилом отсеке (Холл) космической станции «Прогер-67». Станция обесточена, погружена во тьму, а системы жизнеобеспечения работают в аварийном режиме. Связь с Центром отсутствует.

Ваша цель: Добраться до аварийного выхода в ангаре, запустив по пути резервный генератор, чтобы открыть гермодвери.

Кто куда ходит: Игрок перемещается по локациям в поисках предметов и решения головоломок. На станции, кроме вас, никого нет, но автоматика (терминалы, ловушки) и сама среда (холод, темнота) представляют смертельную опасность.

2 Карта локаций

Игрок начинает в центре и должен исследовать локации, чтобы собрать предметы и открыть путь к выходу. Мир меняется по мере выполнения задач.



3 Арсенал предметов (Inventory)

Предметы нужны для взаимодействия с объектами или проверки условий.

Предмет (Item)	Где найти (Location)	Для чего нужен (Effect)
Фонарик (Torch)	Холл (Hall) (в шкафчике)	Позволяет проходить Тёмный коридор
Ключ (Key)	Склад (Storage) (в ящике)	Открывает дверь в Тёмный
Предохранитель (Fuse)	Тёмный коридор (DarkCorridor) (на полу)	Необходим для починки генератора (вместо)
Гаечный ключ (Wrench)	Склад (Storage) (на стеллаже)	Необходим для починки генератора (вместо)
Аптечка (Medkit)	Комната Генератора	Восстанавливает здоровье

4 Архитектура игры (Роли классов)

Это ключевой раздел. Здесь подробно расписано, зачем нужен каждый класс из списка.

4.1 Основные классы

- **Game:** Главный класс-контроллер. Содержит игровой цикл (`while(true)`), ссылку на текущее состояние (`GameState`) и текущую локацию. Запускает парсинг команд и их выполнение.
- **GameState:** Единое хранилище данных. Содержит здоровье игрока (`int Health`), список предметов (`List<string> Inventory`), словарь флагов мира (`Dictionary<string, bool> WorldFlags`), счетчик ходов (`int TurnCount`) и журнал (`List<string> Log`). Только здесь хранятся данные.
- **Location:** Модель локации. Содержит:
 - `Name` (название),
 - `Description` (описание),
 - `List<IInteractable> Interactables` (объекты в локации),
 - `List<GameEventBase> LocationEvents` (события, привязанные к этой локации),
 - `Dictionary<string, string> Exits` (название направления -> название локации). Выходы могут динамически добавляться/удаляться эффектами.
- **Player:** Легковесная прослойка. Может хранить ссылку на `GameState` и делегировать ему изменения (`Player.Health -> GameState.Health`). Нужен для разделения ответственности.

4.2 Интерфейсы

- **ICommand:** Команда пользователя. Имеет метод `void Execute(Game game, string args)`.
- **IInteractable:** Все, с чем можно взаимодействовать в мире. Имеет методы:
 - `string GetName()` (уникальный ID),
 - `string GetDescription()`,
 - `void Interact(GameState state)` (объект сам решает, что делать)
- **ICondition:** Проверка условия. Имеет метод `bool IsMet(GameState state)`.
- **IEffect:** Изменение состояния. Имеет метод `void Apply(GameState state)`.

4.3 Абстрактные классы

- **CommandBase:** Содержит общую логику для команд (например, вывод справки или проверка, что игра не завершена). Все конкретные команды наследуются от него.
- **ConditionBase:** Может содержать общее описание условия для дебага.
- **EffectBase:** Может содержать строку-сообщение о результате выполнения эффекта.
- **GameEventBase:** Базовый класс для событий мира. Содержит:
 - `ICondition TriggerCondition` (когда сработать),
 - `List<IEffect> Effects` (что сделать),
 - `bool IsOneTime` (удалить после первого срабатывания),
 - `bool CheckAndApply(GameState state)`.

4.4 Команды

- LookCommand: Выводит описание текущей локации (название, описание, списки объектов и выходов).
- GoCommand: Проверяет, есть ли запрошенный выход в Location.Exits. Если есть — меняет текущую локацию в объекте Game.
- InteractCommand: Ищет в текущей локации объект Interactable по имени (аргументу) и вызывает его метод Interact().
- InventoryCommand: Проходится по GameState.Inventory и выводит список предметов.
- HelpCommand: Выводит список доступных команд и их краткое описание.

4.5 Условия

- HasItemCondition: Проверяет, есть ли предмет с заданным именем в GameState.Inventory.
- FlagCondition: Проверяет значение флага в GameState.WorldFlags (например, "GeneratorOn" == true).
- HealthCondition: Проверяет здоровье (например, Health < 25).
- AndCondition: Получает в конструктор два других ICondition. Возвращает true, только если оба внутренних условия вернули true.
- OrCondition: Аналогично, возвращает true, если хотя бы одно условие вернуло true.
- NotCondition: Инвертирует результат внутреннего условия.

4.6 Эффекты

- AddItemEffect: Добавляет предмет в GameState.Inventory.
- RemoveItemEffect: Удаляет предмет из инвентаря (если он там есть).
- SetFlagEffect: Устанавливает флаг в GameState.WorldFlags в true или false.
- DamageEffect: Уменьшает GameState.Health на заданную величину.
- HealEffect: Увеличивает здоровье (но не выше максимума).
- LogEffect: Добавляет строку в журнал событий GameState.Log. Эта строка потом может выводиться игроку.
- AddExitEffect: Динамически добавляет новый выход в текущую или указанную локацию.
- ChangeLocationEffect: Принудительно перемещает игрока в другую локацию.

4.7 Объекты

- Chest (Сундук): Реализует Interactable.
 - В конструкторе принимает List<IEffect> effectsOnOpen (например, AddItemEffect для ключа).
 - Может содержать внутренний флаг */text_iOpened..*
- Door (Дверь): Реализует Interactable.
 - Содержит ICondition openCondition.
 - При Interact проверяет условие. Если IsMet() == true, применяет AddExitEffect, чтобы соединить локации.

- NPC / Terminal (Терминал): Содержит Dictionary<string, (ICondition, List<IEffect>)> — фразы-команды. Если игрок пишет, например, "осмотреть терминал", терминал проверяет условие для фразы "осмотреть" и выдает соответствующий текст или применяет эффект.
- Trap (Ловушка): Содержит ICondition triggerCondition (например, всегда true) и List<IEffect> trapEffects (DamageEffect). При активации применяет эффекты и, если ловушка одноразовая, удаляет себя из локации.

4.8 События

- OnEnterLocationEvent: Срабатывает, когда игрок заходит в локацию. Проверяет условие (например, NotCondition на наличие фонарика) и применяет эффекты (урон).
- OnTurnEvent: Событие, которое проверяется в начале каждого хода игрока (перед вводом команды). Можно использовать для счетчиков времени или нарастания угрозы.
- OneTimeEvent: Обычный GameEventBase с флагом IsOneTime = true. После срабатывания удаляется из списка событий локации или мира.

5 Игровые механики (Сценарий)

Как это будет работать в собранном виде. Цифрами обозначены шаги, на которых срабатывают условия и эффекты.

5.1 Механика 1: Тёмный коридор (Наносит урон)

1. Игрок в Холле. У него есть Фонарик (Torch).
2. Игрок вводит go darkcorridor.
3. Система проверяет: выход "darkcorridor" в локации Hall ведет в DarkCorridor.
4. Событие: Срабатывает OnEnterLocationEvent, привязанное к локации DarkCorridor.
 - Условие: NotCondition от HasItemCondition("Torch") (Фонарика нет).
 - Эффект: DamageEffect(15) и LogEffect("Холод пробирает до костей. Вы теряете сознание на минуту...").
5. Если фонарик есть, условие не срабатывает, и игрок проходит без урона.

5.2 Механика 2: Дверь с ключом

1. В локации Hall есть объект Door с ID "Дверь в коридор".
2. Игрок вводит interact "Дверь в коридор".
3. Объект Door вызывает свою логику:
 - Условие: HasItemCondition("Key").
 - Эффект при успехе: AddExitEffect("darkcorridor", "DarkCorridor") в локации Hall + LogEffect("Дверь со щелчком открывается.").
 - Эффект при неудаче: LogEffect("Дверь заперта. Нужен ключ-карта.").

5.3 Механика 3: Генератор (Требует 2 предмета)

1. В GeneratorRoom стоит объект Chest ("Распредщит").
2. Игрок вводит interact "Распредщит".
3. Объект Chest проверяет внутреннее состояние. Если еще не открыт, он проверяет сложное условие (которое внутри объекта может быть реализовано через AndCondition).
 - Условие: AndCondition(HasItemCondition("Fuse"), HasItemCondition("Wrench")).
 - Эффекты: SetFlagEffect("GeneratorOn", true), RemoveItemEffect("Fuse"), RemoveItemEffect("Wrench"), LogEffect("Генератор ожил! Свет моргнул и загорелся. Где-то щелкнул замок двери."), и AddExitEffect("exit", "Exit") в локации DarkCorridor.

5.4 Механика 4: Одноразовая ловушка

1. В Storage на полу лежит объект Trap ("Растяжка").
2. Игрок входит в Storage.
3. Срабатывает OnEnterLocationEvent для этой локации, который проверяет наличие ловушки (или ловушка срабатывает сама, если на неё "наступить" командой interact, что менее логично). Лучше сделать событие:
 - Условие: FlagCondition("TrapArmed") (изначально true).
 - Эффекты: DamageEffect(10), SetFlagEffect("TrapArmed", false), LogEffect("Вы задели растяжку!").

5.5 Механика 5: Несколько концовок

- Плохая концовка: Здоровье игрока (GameState.Health) падает до 0. Игра завершается сообщением о смерти.
- Хорошая концовка: Игрок перемещается в локацию Exit (срабатывает ChangeLocationEffect или просто команда go). В локации Exit есть OnEnterLocationEvent, который проверяет флаг GeneratorOn.
 - Условие: FlagCondition("GeneratorOn").
 - Эффект: LogEffect("Шлюз открыт. Вы покидаете станцию. Это победа!") + завершение игры.
 - Если генератор не включен: LogEffect("Дверь шлюза заблокирована питанием. Нужно запустить генератор.").

6 Система квестов

6.1 Квест 1: "Запуск генератора" (Power Restoration)

6.1.1 Данные квеста

Поле	Значение
Название	"Аварийное питание"
Описание	Запустить резервный генератор, чтобы восстановить питание на станции и открыть доступ к выходу
Состояние	Active (активируется сразу при старте игры)
Награда	Открывается выход + 50 очков выживания

6.1.2 Этапы квеста

1. Найти гаечный ключ (Find Wrench)

- Описание: Обыскать склад в поисках гаечного ключа. Без него не получится открыть щиток генератора.
- Условие: `HasItemCondition("Wrench")`
- Диалог/событие:
 - До выполнения: "На складе должен быть ящик с инструментами. Найти бы его..."
 - При выполнении: "Гаечный ключ найден! Теперь можно открутить защитную панель."

2. Найти предохранитель (Find Fuse)

- Описание: Отыскать целый предохранитель. Старый перегорел, и без нового генератор не запустить.
- Условие: `HasItemCondition("Fuse")`
- Диалог/событие:
 - До выполнения: "Предохранитель должен быть где-то в тёмном коридоре... если только его не сожрала тьма."
 - При выполнении: "Предохранитель цел! Похоже, его можно использовать."

3. Включить генератор (Activate Generator)

- Описание: Использовать ключ и предохранитель, чтобы запустить генератор в GeneratorRoom.
- Условие: `FlagCondition("GeneratorOn")` (устанавливается при взаимодействии с объектом Generator)
- Диалог/событие:
 - До выполнения: "Предохранитель должен быть где-то в тёмном коридоре... если только его не сожрала тьма."
 - При выполнении: "Гул генератора разносится по станции. Свет моргает и загорается!"

6.1.3 Текстовое сопровождение

При обновлении: Система: "Квест 'Аварийное питание' обновлен. Этап 'Найти гаечный ключ' выполнен!"

При завершении: Система: "Квест 'Аварийное питание' выполнен! Доступ к выходу открыт. Получена награда: +50 очков выживания."

6.2 Квест 2: "Путь к спасению" (Escape Route)

6.2.1 Данные квеста

Поле	Значение
Название	"Последний рубеж"
Описание	Добраться до аварийного выхода, преодолев все препятствия.
Состояние	NotStarted (активируется после запуска генератора)
Награда	Финальный ролик/сообщение + запись в достижения

6.2.2 Этапы квеста

Квест становится активным только после выполнения условия: `FlagCondition("GeneratorOn")`. Это демонстрирует, как квесты могут быть связаны между собой.

1. Пройти темный коридор (Cross DarkCorridor)

- Описание: Добраться до выхода через темный коридор, избежав ловушек.
- Игрок находится в локации DarkCorridor И флаг "GeneratorOn" == true
- Особенность: Если игрок попытался пройти коридор до включения генератора, он получает урон, но квест все равно ждет включения генератора.

2. Открыть шлюз (Open Airlock)

- Описание: Найти пульт управления шлюзом и активировать его.
- Условие: FlagCondition("AirlockUnlocked") (устанавливается объектом Terminal в DarkCorridor)

3. Достичь выхода (Reach Exit)

- Описание: Шагнуть в спасательный челнок.
- Условие: Игрок находится в локации Exit
- Финальный эффект:
 - LogEffect("Поздравляем! Вы выбрались со станции!")
 - ChangeLocationEffect("Credits") (теоретическая локация с титрами)

6.2.3 Текстовое сопровождение

Финальное сообщение: КВЕТ ВЫПОЛНЕН: КВЕТ ВЫПОЛНЕН: "Последний рубеж"

Вы выбрались с гибнущей станции! Спасательный челнок отстыковался. Впереди - долгий путь домой...